

Marcar como hecha

Hasta ahora, de AngularJS hemos visto muchas cosas interesantes, sin embargo nos hemos limitado a hacer aplicaciones en las que solo teníamos una ruta y una vista. Sin embargo, cuando se vayan complicando los requisitos de nuestras aplicaciones podremos necesitar ampliar estas posibilidades.

Lo que vamos a aprender ahora es a crear rutas distintas dentro de nuestra aplicación, de modo que tengamos varias URL que muestran vistas distintas. Quizás te preguntes ¿Si AngularJS está especialmente indicado para hacer aplicaciones de una sola página (concepto que se conoce con el nombre de "Single Page Application", SPA), por qué ahora nos muestras la posibilidad de crear varias URL? ¿No era una única página lo que queríamos hacer?

Pues bien, podemos tener ambas cosas. Una aplicación de una sola página, pero que es capaz de representar URL distintas, **simulando lo que sería una navegación** a través de la aplicación, pero sin salirnos nunca de la página inicial.

Esto nos sirve para varias cosas, entre otras:

- Memorizar rutas profundas dentro de nuestra aplicación. Podemos contar con enlaces que nos lleven a partes internas (deeplinks), de modo que no estemos obligados a entrar en la aplicación a través de la pantalla inicial.
- Eso facilita también el uso natural del sistema de favoritos (o marcadores) del navegador, así como el historial. Es decir, gracias a las rutas internas, seremos capaces de guardar en favoritos un estado determinado de la aplicación. A través del uso del historial del navegador, para ir hacia delante y atrás en las páginas, podremos navegar entre pantallas de la aplicación con los botones del navegador.
- Mantener vistas en archivos independientes, lo que reduce su complejidad y administrar los controladores que van a facilitar el procesamiento dentro de ellas.

Lo que generaremos dará distintas url que añadirán a la página un `"/#nombre"` Siendo nombre una ruta, o pseudoruta, del modo en que utilizábamos enlaces anchor a la misma página.

Como sabemos, la "almohadilla", sirve para hacer rutas a anclas internas, zonas de una página.

Cuando le pidamos al navegador que acceda a una ruta creada con `"#"` éste no va a recargar la página, yéndose a otro documento. Lo que haría es buscar el ancla que corresponda y mover el scroll de la página a ese lugar.

En el caso de AngularJS no habrá un movimiento de scroll, pues con Javascript se detectará el cambio de ruta en la barra de direcciones para intercambiar la vista que se está mostrando.

Por ello, volviendo a la pregunta de antes: "si la posibilidad de crear varias rutas dentro de una aplicación contradice el sistema de Single Page Application", observamos que realmente no son páginas distintas, sino que es la misma página.

Lo que estamos haciendo es "simular" la navegación por varias URL cuando realmente es la misma, con enlaces internos.

Instalar ngRoute

Destaco este aspecto, ya que todavía en las versiones actuales, Angular pregunta si queremos el módulo ngRoute.

El módulo ("module" en la terminología anglosajona de Angular) ngRoute es un potente paquete de utilidades para configurar el enrutado y asociar cada ruta a una vista y un controlador, tal como hemos dicho. Sin embargo este módulo no está incluido en la distribución de base de Angular, sino que en caso de pretender usarlo tenemos que instalarlo y luego inyectarlo como dependencia en el módulo principal de nuestra aplicación.

Como no lo tenemos en el script básico de Angular, debemos instalarlo "a mano". lo conseguimos incluyendo el script del código Javascript del módulo ngRoute.

El que utilizaremos es

```
<script src = "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular-route.min.js">
```

Inyección de dependencias

El segundo paso sería inyectar la dependencia con ngRoute en el módulo general de nuestra aplicación. Esto lo hacemos en la llamada al método `module()` con el que iniciamos cualquier programa AngularJS, indicando el nombre de las dependencias a inyectar en un array.

```
angular.module("app", ["ngRoute"])
```

Hasta ahora este array de dependencias, cuando llamábamos a `angular.module()`, estaba siempre vacío. A medida que se compliquen nuestras aplicaciones podremos necesitar inyectar más cosas. Por ejemplo, las directivas creadas por

?

terceros desarrolladores también las inyectarás de esta manera en tu módulo principal de la aplicación.

Configurar el sistema de enrutado con \$routeProvider

El sistema de enrutado de AngularJS nos permite configurar las rutas que queremos crear en nuestra aplicación de una manera declarativa. Aunque sea un componente bastante complejo internamente, podemos configurarlo de una manera ciertamente sencilla.

\$routeProvider tiene un par de métodos. El primero es when(), que nos sirve para indicar qué se debe hacer en cada ruta que se desee configurar, y el método otherwise() que nos sirve para marcar un comportamiento cuando se intente acceder a cualquier otra ruta no declarada.

Esta configuración se debe realizar dentro del método config(), que pertenece al modelo. De hecho, solo podemos inyectar \$routeProvider en el método config() de configuración.

```
angular.module("app", ["ngRoute"])
```

```
.config(function($routeProvider){
```

```
//configuración y definición de las rutas
```

```
});
```

Las rutas las configuras por medio del método when() que recibe dos parámetros. Por un lado la ruta que se está configurando y por otro lado un objeto que tendrá los valores asociados a esa ruta. De momento vamos a conocer solo éstos que serían los fundamentales para empezar.

- "controller", para indicar el controlador
- "controllerAs", el nombre con el que se conocerá el scope dentro de esa plantilla
- "templateUrl", el nombre del archivo, o ruta, donde se encuentra el HTML de la vista que se debe cargar cuando se acceda a la ruta.

Veamos con código completo

el html (fijaros que no es necesario ya declarar la ng-app en la etiqueta html:

```
<html>
  <head>
    <title>Angular JS Vistas con routing</title>
    <script src = "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
    <script src = "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular-route.min.js">
    </script>
    <script src="angcod.js"></script>
  </head>

  <body>
    <h2>Ejemplo Routing</h2>
    <div ng-app = "mainApp">
      <p><a href = "#addStudent">Añade estudiante</a></p>
      <p><a href = "#viewStudents">Ver estudiantes</a></p>
      <div ng-view></div>

      <script type = "text/ng-template" id = "addStudent.htm">
        <h2> Añadir estudiante </h2>
        {{message}}
      </script>

      <script type = "text/ng-template" id = "viewStudents.htm">
        <h2> Ver estudiantes </h2>
        {{message}}
      </script>
    </div>
```

el js, que lo he llamado angcod.js

```
var mainApp = angular.module("mainApp", ['ngRoute']);

mainApp.config(['$routeProvider', function($routeProvider) {
  $routeProvider

    .when('/addStudent', {
      templateUrl: 'addStudent.htm', controller: 'AddStudentController'
    })
    .when('/viewStudents', {
      templateUrl: 'viewStudents.htm', controller: 'ViewStudentsController'
    })
    .otherwise ({
      redirectTo: '/addStudent'
    });
}]);

mainApp.controller('AddStudentController', function($scope) {
  $scope.message = "Página que se utiliza cuando se añade un estudiante";
});

mainApp.controller('ViewStudentsController', function($scope) {
  $scope.message = "Página donde se muestran los estudiante";
});
```

Echa un vistazo a los enlaces que hay dentro del html. Esos son los enlaces que provocarían la navegación en el enrutado definido en la anterior configuración. Observarás que los href de los enlaces están asociados a las mismas rutas definidas en el \$routeProvider. Por tanto, pulsando cada enlace Angular nos mostrará la vista indicada en la declaración que hemos indicado en cada when del \$routeProvider.

Tienes también que fijarte en la división DIV que hay más abajo que tiene la directiva ngView. En principio no necesitas indicar más cosas como valor de esa directiva. AngularJS ya sabe que las vistas se desplegarán en ese contenedor. Y lo hacemos con sendas etiquetas script, que se visualizarán únicamente en cada uno de los casos.

Así pues, nos interesa tanto ver que cambia la url, del modo que lo haría en una navegación normal, pero no está accediendo al servidor y realizando una solicitud, que es lo que en AngularJS, que todavía sigue utilizándose mucho, se intenta.